

PLANO DE TESTE

SmileCorp — Sistema de Gerenciamento Odontológico

Time
Eduardo · Erick · Samara

Plataforma
Web / App · Java 21 · Next.js · TypeScript

Frameworks de Teste
Playwright · JUnit 5 + Mockito · Cucumber
7.21

Repositório
github.com/erick-alexandro/SmileCorp

1. Identificação e Contexto

Nome do Projeto	SmileCorp
Objetivo do Sistema	Solução de gerenciamento para clínicas odontológicas. Permite dentistas e administradores foco no atendimento com controle total sobre recursos da clínica.
Público-Alvo	Clínicas de Odontologia
Plataforma	Web / App
Repositório	https://github.com/erick-alexandro/SmileCorp
Time	Eduardo, Erick, Samara

2. Ambiente e Ferramentas

Sistema Operacional	Windows 11
Linguagem / Runtime	Java 21 · Next.js · TypeScript
IDE	VS Code
Build	Maven
Testes Unitários	JUnit 5 + Mockito
BDD	Cucumber 7.21
Banco de Dados	Neon (PostgreSQL via Spring Boot + JPA + Flyway)

3. Objetivo do Teste

Validar a qualidade e estabilidade das principais funcionalidades do SmileCorp via testes end-to-end com Playwright, cobrindo os fluxos críticos de autenticação, agendamento, cadastro de pacientes, procedimentos, fornecedores e controle de estoque. Os testes garantem que a interface renderize corretamente, que as validações de formulário funcionem, que os estados de carregamento/vazio sejam exibidos e que as operações de busca, filtro e interação com modais estejam operacionais nos 3 browsers (Chromium, Firefox, WebKit).

Metas de Cobertura

- $\geq 90\%$ das páginas críticas cobertas por testes E2E (auth, agendamento, pacientes, procedimentos, fornecedores, estoque)
- $\geq 80\%$ dos estados de interface validados (loading, empty, erro de validação, dados renderizados, busca sem resultados)
- $\geq 80\%$ das validações de formulário testadas (campos obrigatórios, formato de e-mail, CNPJ, CPF)
- $\geq 90\%$ das operações CRUD frontend verificadas (cadastro, edição, exclusão, listagem com filtros)
- 100% dos componentes modais testados (abertura, campos, botões, fechamento)

4. Escopo

Categoria	Em Escopo	Fora do Escopo
Funcionalidades	• Autenticação (login, logout) • Agendamento (calendário, criação, filtros) • Cadastro de Pacientes (CRUD, perfil completo) • Procedimentos (CRUD, filtros, materiais) • Fornecedores (CRUD, validação CNPJ/CPF) • Estoque (entrada/saída/ajuste, histórico) • Financeiro (receitas/despesas, parcelas) • Prontuário e Gestão de Equipe	• Dashboard e relatórios gerenciais • Notificações push e chat interno • Portal do paciente (público) • Agendamento online (público) • Integração com operadoras de plano • Testes de performance/carga • Testes de acessibilidade (WCAG) • Testes de segurança (OWASP)
Integrações	• Frontend ↔ Backend via Next.js proxy (/api/proxy/*) • Neon Auth (autenticação + organizações) • PostgreSQL via Spring Boot + JPA + Flyway • Cliente HTTP nativo (fetch) para API calls	• Sistemas externos (CNS, TISS, BPA) • Gateway de pagamento (Pix, cartão) • Correio/CEP automático • WhatsApp/Telegram para lembretes • Exportação contábil (SPED)

5. Estratégia de Testes

Tipo	Objetivo	Escopo	Ferramenta	Responsável	Saída
Unitário	Validar regras de negócio e serviços isoladamente	8 serviços: Paciente, Fornecedor, Agendamento, Produto, Procedimento, Profissional, Transação, Conta Fixa	JUnit 5 + Mockito	Backend	Relatório Maven
E2E (Playwright)	Validar fluxos completos da interface, renderização e estados	5 páginas · 53 testes · Chromium, Firefox, WebKit	Playwright	Frontend	HTML + JUnit XML
BDD	Validar regras de negócio em linguagem Given/When/Then	Fornecedor, Agendamento, Produto (preço médio ponderado)	Cucumber 7.21	Backend	Report BDD

6. Rastreabilidade (Requisitos × Testes)

ID	Requisito / Funcionalidade	Prioridade	Issue	Status
REQ-001	Autenticação — login com e-mail e senha, validação de campos e formato	Alta	#001	Aprovado
REQ-002	Fornecedores — CRUD com validação CNPJ/CPF, busca e filtro por status	Alta	#002	Aprovado
REQ-003	Agendamento — calendário com navegação entre períodos e filtros	Média	#003	Aprovado

7. Casos de Teste Planejados

TC-001 E2E Alta	Login — exibir formulário com campos e botão	
Pré-condição	Entrada	Resultado esperado
Navegar para /auth/sign-in	Aguardar o carregamento completo da página e inspecionar os elementos renderizados	Campos de e-mail, senha e botão submit visíveis; label 'E-mail' visível

TC-002 E2E Alta	Login — rejeitar e-mail inválido	
Pré-condição	Entrada	Resultado esperado

Navegar para /auth/sign-in	Digitar 'invalid-email' no campo e-mail	checkValidity() retorna false
----------------------------	---	-------------------------------

TC-003 E2E Alta	Fornecedores — validação de campos obrigatórios no cadastro	
Pré-condição	Entrada	Resultado esperado
/inventory/suppliers vazio, clicar no botão '+'	Clicar em 'Cadastrar' sem preencher campos	8 mensagens de erro visíveis: Nome, CNPJ/CPF, Telefone, Email, Rua, Número, Bairro, Cidade

TC-004 E2E Média	Fornecedores — filtrar por status 'Ativo'	
Pré-condição	Entrada	Resultado esperado
1 fornecedor ativo + 1 inativo na lista	Selecionar 'Ativo' no filtro de status	Apenas o fornecedor ativo visível na tabela

TC-005 BDD Alta	Fornecedores — cadastrar novo fornecedor com dados válidos	
Pré-condição	Entrada	Resultado esperado
org-cucumber-3 definida como organização ativa	Nome: Dental Plus Ltda · CNPJ: 11222333000155 · Email: contato@dentalplus.com	Fornecedor salvo com todos os dados informados: nome, CNPJ, telefone, email, endereço

TC-007 BDD Média	Produtos — reabastecimento calcula preço médio ponderado	
Pré-condição	Entrada	Resultado esperado
Produto 'Resina' com preço R\$ 45,00 e estoque 10 unidades	Reabastecer 5 unidades a R\$ 40,00 cada	Novo estoque = 15 unidades · Preço médio ponderado = R\$ 43,33

8. Dados de Teste

ID	Conjunto	Descrição	Como Criar	Onde Armazenar	Observações
DT-001	Fornecedores mockados	2 fornecedores (1 ativo, 1 inativo) para testes de listagem, busca e filtro	page.route('**/api/proxy/suppliers').fulfill() com JSON	Inline em suppliers.spec.ts via route.fulfill()	CNPJ ativo deve ser válido (11222333 000181)
DT-002	Produto com movimentações	1 produto com 2 movimentos de entrada para testar histórico, preço médio e badge de estoque baixo	page.route('**/api/proxy/produtos**').fulfill() com JSON	Inline em products.spec.ts	currentQuantity < minStock para exibir badge 'ESTOQUE BAIXO'
DT-003	Procedimentos com filtros	2 procedimentos (categorias distintas) para busca, filtros e tabela	page.route('**/api/proxy/procedimentos**').fulfill() com JSON	Inline em procedures.spec.ts	ativo deve ser booleano; materiais pode ser array vazio